Segundo Cerebro

Un asistente RAG 100 % local sobre un repositorio de notas personales — Documento técnico

Octubre 2026

1. Qué es este proyecto

“Segundo Cerebro” es un asistente de consulta sobre mis propios documentos. Tomo mis notas y documentos personales (guardados en un repositorio local de notas en Markdown), los convierto en vectores, los guardo en una base de datos vectorial y los consulto en lenguaje natural desde un chat web. Un modelo de lenguaje responde usando solo los fragmentos recuperados y cita de dónde sacó cada cosa.

Todo corre en un computador pequeño propio, fuera de cualquier nube. Ninguna pregunta ni ningún documento sale a Internet. Esa es la regla número uno del proyecto y la razón de la mayoría de las decisiones técnicas que se describen más abajo.

El proyecto nació con un objetivo práctico: dejar de buscar a mano “en qué documento decía tal cosa” y poder preguntar directamente, con la respuesta respaldada por citas verificables.

1.1 Por qué RAG y no fine-tuning

Al principio evalué dos caminos para que un modelo “supiera” de mis documentos:

- **Fine-tuning:** reentrenar un modelo con mis textos. Caro en cómputo, lento de actualizar (cada documento nuevo exige reentrenar) y propenso a que el modelo “recuerde mal” sin poder citar la fuente.
- **RAG (Retrieval-Augmented Generation):** dejar el modelo tal cual y, en cada pregunta, buscar primero los fragmentos relevantes y entregárselos como contexto. Se actualiza al instante (basta indexar el documento nuevo), funciona con modelos pequeños y cada respuesta puede citar su fuente.

Para un repositorio que cambia todas las semanas y para un equipo sin GPU, RAG era la única opción razonable.

2. Arquitectura general

El sistema tiene cinco piezas que se comunican dentro de la misma máquina:

1. **Repositorio de notas**, sincronizado entre el notebook y el servidor con una herramienta de sincronización de archivos.
2. **Conversión de documentos:** scripts que transforman `.docx` y `.pdf` a texto plano.
3. **Indexador:** un workflow de n8n que corta los textos en fragmentos, los convierte en vectores y los guarda en Qdrant.
4. **API de consulta:** otro workflow de n8n que recibe la pregunta, busca en Qdrant, arma el contexto y llama al modelo de lenguaje.
5. **Front web:** una página HTML de un solo archivo con el chat, las citas y un panel de estado.

Flujo resumido:

```
Notebook <-> sincronización de archivos <-> Servidor
    |
    v
convertir docx/pdf -> txt
    |
    v
Indexador (n8n): chunking -> embeddings -> Qdrant
    |
    v
Chat web -> API Preguntar (n8n) -> Qdrant (buscar) -> LLM (responder con citas)
```

2.1 Hardware y plataforma

Componente	Elección	Motivo
Equipo	Mini PC de oficina usado, CPU de 4 núcleos, 32 GB de RAM, sin GPU	Barato, silencioso, bajo consumo. Es un homelab, no un centro de datos.
Sistema operativo	Ubuntu Server	Estable, sin interfaz gráfica, fácil de administrar por SSH.
Contenedores	Docker	Aísla cada servicio (n8n, Qdrant, nginx, dashboard).
Acceso remoto	Red privada cifrada	Puedo usar el chat desde el notebook o el teléfono sin abrir puertos a Internet.

La falta de GPU es la restricción que más pesa: define qué modelos puedo usar y cuánto demora cada respuesta.

2.2 Stack de software

Función	Herramienta
Orquestación de workflows	n8n (self-hosted)
Base de datos vectorial	Qdrant
Modelo de lenguaje (razonamiento)	Modelo abierto local, en dos tamaños (7B y 3B)
Modelo de embeddings	Modelo abierto local de embeddings (768 dimensiones)
Conversión de documentos	LibreOffice headless y pdftotext
Sincronización de archivos	Herramienta de sincronización peer-to-peer
Front	HTML + JavaScript sin dependencias, servido por nginx
Tareas periódicas	cron

3. La ingesta: de documento a vector

Esta es la parte del sistema que más tiempo me tomó dejar bien y la que más se parece a cualquier pipeline de RAG.

3.1 Conversión de documentos

Solo se indexan archivos `.md` y `.txt`. Los `.docx` y `.pdf` se convierten primero a `.txt` con scripts que corren cada pocos minutos fuera de n8n. Cada original queda acompañado de su versión en texto.

Decidí convertir fuera de n8n después de un incidente: el nodo de extracción de PDF entregaba el texto conservando la extensión `.pdf`, y un script de limpieza confundió ese texto con el archivo original y lo borró. Lección: la conversión es un paso aparte, y n8n solo ve archivos de texto.

3.2 Chunking

Cada documento se parte en fragmentos (“chunks”) de tamaño acotado. Un fragmento demasiado largo diluye el significado del vector y no cabe en el contexto del modelo; uno demasiado corto pierde el hilo. Los fragmentos que llegan al modelo se recortan a unos 900 caracteres.

Cada fragmento se guarda con metadatos: nombre del archivo, ruta, carpeta, número de fragmento, total de fragmentos y un hash del contenido del documento.

3.3 Embeddings con prefijos

El modelo de embeddings que uso es **asimétrico**: espera un prefijo distinto según si el texto es un documento que se guarda o una pregunta que se busca.

- Al indexar: `search_document: <texto del fragmento>`
- Al consultar: `search_query: <pregunta>`

No usar los prefijos degrada la calidad de la búsqueda de forma silenciosa: todo “funciona”, pero los resultados son peores. Cuando adopté los prefijos tuve que reindexar el repositorio completo, porque los vectores antiguos ya no eran comparables con las nuevas consultas.

3.4 Almacenamiento en Qdrant

- Colección `documentos`: vectores de 768 dimensiones, distancia coseno.
- Identificador de cada punto: hash de `ruta + número de fragmento`. Así, reindexar un documento sobreescribe sus propios puntos en vez de duplicarlos.
- Índices sobre los metadatos (ruta, carpeta, hash) para poder filtrar y borrar por documento.
- Índice de texto completo sobre el campo `texto`, que se usa para la búsqueda híbrida (sección 4.2).

3.5 Detección de cambios

El indexador corre cada 10 minutos. Para no reprocesar todo el repositorio cada vez, compara el hash del contenido de cada archivo con el que tiene guardado en Qdrant. Si cambió, borra los fragmentos viejos y vuelve a indexar.

La primera versión comparaba el **tamaño del archivo**. Resultó ser un error: n8n entrega el tamaño redondeado (por ejemplo “739 kB”), así que una edición pequeña era invisible y el documento nunca se reindexaba. Comparar por hash resolvió el problema de raíz.

4. La consulta: de pregunta a respuesta con citas

4.1 Flujo de una pregunta

1. El usuario escribe en el chat web.
2. La pregunta se convierte en vector con el prefijo `search_query:`.
3. Se buscan los 12 fragmentos más cercanos en Qdrant.
4. Se aplica búsqueda híbrida (4.2) y se eligen los 5 mejores.
5. Los 5 fragmentos, recortados, se arman en un prompt junto con la pregunta.
6. El modelo de lenguaje responde citando los fragmentos como `[1]`, `[2]`, etc.
7. El front muestra la respuesta y, debajo, las fuentes con enlaces que abren el documento original en el editor de notas.

El modelo recibe instrucciones estrictas: responder solo con lo que está en el contexto y decir explícitamente cuando la información no está. Prefiero un “no lo sé” a una respuesta inventada.

4.2 Búsqueda híbrida

La búsqueda puramente semántica a veces falla con términos exactos: códigos de documento, siglas, nombres propios. Un código como `PRO-005` no tiene “significado” que un vector capture bien.

La solución fue combinar la búsqueda vectorial con una búsqueda por palabras exactas usando el índice de texto completo de Qdrant, y fusionar ambos rankings. Hubiera preferido usar vectores dispersos (sparse vectors) nativos, pero Qdrant no permite agregarlos a una colección creada con un vector único sin recrearla. El índice de texto completo fue el compromiso para no reindexar todo.

4.3 Latencia

Sin GPU, el modelo de 7B demora alrededor de 100 segundos por respuesta con 5 fragmentos de contexto. La versión 3B es bastante más rápida a costa de calidad. El chat permite elegir entre ambos. Para mi uso (pocas preguntas al día, respuestas que necesito correctas) la espera es aceptable; para un uso intensivo habría que invertir en GPU o aceptar un modelo más pequeño.

5. Clasificación automática de documentos nuevos

Además de responder preguntas, el sistema ordena los documentos que llegan. Hay una carpeta de “pendientes por clasificar”; un script la revisa cada 5 minutos y propone en qué carpeta del repositorio debería ir cada archivo.

Funcionamiento:

1. Existe una segunda colección en Qdrant, `carpetas`, con un vector por carpeta calculado a partir de una descripción escrita de qué contiene cada una.
2. El documento nuevo se vectoriza y se compara contra las carpetas. Se toman las 3 más cercanas.
3. Si la diferencia de puntaje entre la primera y la segunda es menor a 0,08, el caso es ambiguo y se le pide al modelo de lenguaje que decida leyendo el documento.
4. La propuesta va a una **bandeja de aprobación** visible en el chat. Nada se mueve sin que yo apruebe (o corrija) la carpeta.
5. Una vez aprobado, el script mueve el documento junto con su versión convertida.

Un hallazgo de calibración: los puntajes de similitud quedaban entre 0,67 y 0,71, con márgenes muy pequeños entre carpetas, lejos del umbral de 0,80 que había fijado para automatizar. La tentación era bajar el umbral. Lo correcto fue mejorar las descripciones de las carpetas, que eran demasiado genéricas; el problema estaba en los datos de referencia, no en el algoritmo.

6. Seguridad y privacidad

Principios aplicados:

- **Nada sale del servidor.** Modelos, base vectorial, orquestador y front viven en la misma máquina.
- **Mínimo privilegio.** El chat solo puede registrar decisiones; quien mueve archivos es el script clasificador. Los contenedores no son alcanzables desde la red local ni desde Internet, solo desde la red privada.
- **Leer → proponer → ejecutar con confirmación.** Toda acción que modifica el repositorio pasa por aprobación humana.
- **Respaldos diarios** de la base vectorial y de la base de datos de n8n, con prueba de restauración hecha.
- **Control de versiones** del repositorio con git, con commit automático cada 10 minutos, además del versionado de la herramienta de sincronización.

6.1 La lección más importante

Durante los primeros meses, la parte de “razonamiento” del sistema usaba un modelo en la nube por API. En una auditoría descubrí que, por cada pregunta, se estaban enviando hasta 25 fragmentos de mis documentos a ese servicio externo. Nadie lo había notado porque todo funcionaba.

La migración a modelos locales fue la consecuencia directa. Y la lección es general: **si un sistema de IA toca datos propios, hay que auditar qué sale y hacia dónde, no asumirlo.** Que funcione no significa que sea privado.

7. Lecciones aprendidas

1. **Los prefijos de embeddings importan.** Un modelo asimétrico sin prefijos da resultados peores sin avisar.
2. **Comparar por hash, no por tamaño.** Los metadatos redondeados esconden cambios.
3. **Los loops en n8n deben cerrarse.** El último nodo del cuerpo de un “Loop Over Items” debe conectarse de vuelta al loop; si no, procesa solo el primer lote y termina en silencio. Me costó una ejecución colgada de 6 horas descubrirlo.
4. **No hay `fetch` en los nodos Code de n8n.** Hay que usar `this.helpers.httpRequest`. Un `fetch` dentro de un try/catch falla sin que se note; mi primera búsqueda híbrida no funcionó durante

meses por eso.
5. **n8n guarda los datos de las ejecuciones exitosas.** Sin configurar la limpieza automática, el historial llenó el disco.
6. **Convertir PDFs fuera del orquestador.** Un nodo que devuelve texto con extensión `.pdf` es una trampa para cualquier script posterior.
7. **Las limitaciones de la base vectorial se deciden al crear la colección.** Sin vectores con nombre no hay vectores dispersos después.
8. **Un resultado matemáticamente correcto no es necesariamente útil.** Un clasificador con puntajes “razonables” puede estar lejos del umbral que necesita la operación; la solución fue mejorar los datos de referencia, no relajar el criterio.
9. **Auditar qué sale a Internet.** Ver sección 6.1.
10. **Preparar el reinicio automático.** Después de un corte de luz, el servidor no volvió solo hasta configurar la recuperación en BIOS.

8. Estado actual y pendientes

El plan original tenía cinco puntos:

1. Seguridad y limpieza de servicios huérfanos — hecho.
2. Control de versiones del repositorio — hecho.
3. Metadatos en Qdrant y reconciliación automática (borrar vectores de archivos que ya no existen) — hecho.
4. Clasificador con bandeja de aprobación y RAG 100 % local — hecho.
5. Generador de borradores: dejar un pedido en una carpeta, que el sistema reúna el contexto del repositorio y produzca un borrador — pendiente.

Otros pendientes: calibrar el clasificador con un par de semanas de registros, fijar versiones de las imágenes Docker en vez de usar `latest`, y evaluar streaming de la respuesta para que la espera de 100 segundos se sienta menos.

9. Glosario breve

- **RAG:** técnica que combina búsqueda de información con generación de texto. El modelo responde a partir de fragmentos recuperados, no solo de lo que “recuerda”.
- **Embedding:** representación numérica (vector) de un texto, que permite comparar significados midiendo distancias.
- **Chunk:** fragmento de un documento, unidad mínima que se vectoriza y se recupera.
- **Base de datos vectorial:** base de datos optimizada para buscar los vectores más parecidos a uno dado.
- **Distancia coseno:** medida de similitud entre dos vectores basada en el ángulo entre ellos.
- **Búsqueda híbrida:** combinación de búsqueda semántica (vectores) y búsqueda por palabras exactas.
- **Payload:** metadatos guardados junto a cada vector (archivo, ruta, fragmento, etc.).